

Lerneinheit 2 · Die Suche von Hand und was sie kostet

Version 1.0

Lernziele

Nach dieser Lerneinheit können Sie ...

- die semantische Suche in reinem Python selbst implementieren und ihre Bausteine mit eigenen Tests absichern,
- Daten und Verhalten in einer einfachen Klasse bündeln und vorbereitende Rechnungen wiederverwenden,
- begründen, warum numerische Schleifen in reinem Python langsam sind, und Container nach ihren Zugriffskosten auswählen,
- erklären, warum Ergebnisse mit Fließkommazahlen mit einer Toleranz verglichen werden.

In der letzten Lerneinheit lief die semantische Suche als fertige NumPy-Funktion, und Sie haben mit ihren Ergebnissen gearbeitet. Heute schreiben Sie die Rechnung dahinter selbst, in reinem Python und Schritt für Schritt. Danach bündeln Sie Daten und Funktionen in einer Klasse und untersuchen, warum diese Fassung so viel langsamer ist als NumPy. Auf der Landkarte stehen wir damit weiter auf der obersten Station, bei Python selbst.

Vorausgesetzt wird Teil 1 des Python-Grundkurses aus der letzten Lerneinheit. Tupel, Dictionaries und Sets führt dieses Notebook nur so weit ein, wie es sie braucht. In voller Breite behandelt sie Teil 2 des Grundkurses, den Sie im Anschluss an diesen Termin bearbeiten.

An mehreren Stellen stehen Boxen mit dem Titel „Vertiefung“. Sie beantworten Fragen für alle, die es genau wissen wollen. Im Termin überspringen wir diese Boxen, und ihr Inhalt ist nicht prüfungsrelevant.

Inhalt

- 2.1 Aufwärmen M1
- 2.2 Container in Kürze M1
- 2.3 Die Suche von Hand M1
 - 2.3.1 Die Länge eines Vektors
 - 2.3.2 Normalize
 - 2.3.3 Compare: das Skalarprodukt
 - 2.3.4 Warum normalisieren?
 - 2.3.5 Select: alles zusammensetzen
- 2.4 Klassen: die Suche als Objekt M1
- 2.5 Was kostet Python? M2
 - 2.5.1 Erst schätzen, dann messen
 - 2.5.2 Das Laufzeitmikroskop
 - 2.5.3 Was kostet ein Container?
- 2.6 Zahlen im Rechner M2

2.1 Aufwärmen

M1 · Python-Grundlagen

Drei kurze Schnipsel aus dem Stoff von Grundkurs Teil 1. Sagen Sie jeweils die Ausgabe vorher und führen Sie die Zelle erst danach aus.

Kurz überlegen

Was gibt die folgende Zelle aus?

```
total = 0
for x in [3, 4, 5]:
    if x > 3:
        total += x
print(total)
```

Kurz überlegen

Was gibt die folgende Zelle aus?

```
def double(x):
    print(x * 2)

y = double(4)
print(y)
```

Kurz überlegen

Was gibt die folgende Zelle aus?

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

2.2 Container in Kürze

M1 · Python-Grundlagen

Die Kursdaten bestehen aus Listen, Tupeln und Dictionaries. Bevor wir rechnen, lernen Sie diese Container so weit kennen, dass Sie die Daten lesen können. Die nächsten beiden Zellen stellen die Daten bereit: dieselben 400 Wikipedia-Absätze mit ihren Vektoren und dieselben acht Beispielanfragen wie in der letzten Lerneinheit.

```
# Vorgabe: Daten laden – einfach ausführen
import json, math, sys, time
import numpy as np      # heute nur zum Laden der Vektordateien und für einen Vergleich

① with open("data/appetizer_passages.json", encoding="utf-8") as f:
②     passages = json.load(f)["passages"]                # 400 Absätze
    with open("data/appetizer_queries.json", encoding="utf-8") as f:
        queries = json.load(f)["queries"]                # 8 Beispielanfragen
③ vectors = np.load("data/appetizer_embeddings.npy").tolist() # 400 Listen mit je 384 Zahlen
    query_vectors = np.load("data/appetizer_query_embeddings.npy").tolist() # 8 Listen mit je 384 Zahlen

print(len(passages), "Absätze,", len(vectors[0]), "Zahlen je Vektor,", len(queries), "Beispielanfragen")
```

① `with open(...) as f` öffnet die Datei und schließt sie am Ende des eingerückten Blocks wieder; `encoding="utf-8"` legt die Zeichenkodierung fest, in der die Umlaute gespeichert sind

② `json.load(f)` liest den Inhalt der Datei und baut daraus Dicts und Listen

③ `np.load` liest eine `.npy`-Datei als NumPy-Array, `.tolist()` macht daraus verschachtelte Python-Listen mit Python-Zahlen

400 Absätze, 384 Zahlen je Vektor, 8 Beispielanfragen

Die Vektoren liegen in `.npy`-Dateien, dem Dateiformat von NumPy für Zahlenblöcke. Wie NumPy solche Blöcke speichert, ist Thema der nächsten Lerneinheit. Heute wandeln wir sie gleich beim Laden in Python-Listen um und rechnen ab hier ohne NumPy. `vectors` ist eine Liste aus 400 Listen, jede innere Liste ist der Vektor eines Absatzes. `vectors[0]` gehört zu `passages[0]`, `vectors[1]` zu `passages[1]` und so weiter. Für die Anfragen gilt dasselbe mit `query_vectors` und `queries`.

Tupel. Die Paare aus Titel und Ähnlichkeitswert, mit denen Sie im letzten Lab gearbeitet haben, heißen allgemein Tupel, hier sind es 2-Tupel. Ein Tupel fasst mehrere Werte zu einem festen Paket zusammen. Der Zugriff über die Position funktioniert wie bei einer Liste, ändern lässt sich ein Tupel nach dem Anlegen aber nicht mehr: Der Versuch `hit[1] = 0.9` endet mit einem `TypeError`. In C++ entsprechen dem `std::pair` und `std::tuple`.

```
① hit = ("San Diego", 0.831)           # ein Treffer: Titel und Ähnlichkeitswert
② title, score = hit
   print(title, score)
③ print(hit[0], len(hit))
```

- ① runde Klammern mit Komma bilden ein Tupel
- ② das Auspacken verteilt die Werte des Tupels auf zwei Namen, wie im letzten Lab
- ③ der Zugriff über die Position und `len` funktionieren wie bei einer Liste

```
San Diego 0.831
San Diego 2
```

Dictionary. Ein Dictionary, kurz Dict, ordnet Schlüsseln Werte zu. Jeder Absatz der Kursdaten ist ein Dict mit den Schlüsseln `id`, `title` und `text`. Zugegriffen wird über den Schlüssel in eckigen Klammern. Fehlt der Schlüssel, bricht der Zugriff mit einem `KeyError` ab. In C++ entspricht dem `std::map`. Falls Sie damit schon gearbeitet haben, ist ein Unterschied wichtig: Dort legt `m[key]` bei einem fehlenden Schlüssel stillschweigend einen Eintrag an, in Python geschieht das beim Lesen nie.

```
p = passages[0]                       # der erste Absatz der Kursdaten
① print(p["title"])
② print(list(p.keys()))
③ print("title" in p)
④ print(p.get("author", "unbekannt"))
⑤ my_hit = {"title": "San Diego", "score": 0.831}   # ein Treffer mit benannten Feldern
   print(my_hit["score"])
```

- ① der Schlüssel steht in eckigen Klammern, wie ein Index, nur mit Namen statt Position
- ② `keys()` liefert alle Schlüssel eines Dicts, `list(...)` macht daraus eine Liste
- ③ `in` prüft bei einem Dict, ob es den Schlüssel gibt
- ④ `get` liefert bei einem fehlenden Schlüssel den angegebenen Ersatzwert statt eines Fehlers
- ⑤ geschweifte Klammern mit `Schlüssel: Wert` legen ein eigenes Dict an

```
Aufzugsanlage
['id', 'title', 'text']
True
unbekannt
0.831
```

Lernfragen 2.2 #1

1. Der erste Absatz hat den Titel „Aufzugsanlage“. Welche der beiden Prüfungen liefert `True`: `"title" in passages[0]` oder `"Aufzugsanlage" in passages[0]`? Begründen Sie Ihre Antwort.

Set. Ein Set ist eine Menge: Jeder Wert kommt darin höchstens einmal vor, und eine Reihenfolge gibt es nicht. Damit lassen sich Duplikate entfernen. Die nächste Zelle sammelt die Titel aller Absätze und bildet daraus ein Set. Das Ergebnis korrigiert nebenbei eine Aussage aus der letzten Lerneinheit: Die 400 Absätze stammen nicht aus 400 verschiedenen Artikeln, manche Artikel sind mit mehreren Absätzen vertreten.

```

titles = []
for p in passages:
    titles.append(p["title"])           # die Titel aller 400 Absätze
① unique_titles = set(titles)
    print(len(titles), "Absätze aus", len(unique_titles), "verschiedenen Artikeln")
② print("San Diego" in unique_titles)

```

- ① `set(...)` bildet aus einer Liste ein Set, Duplikate fallen dabei weg
 ② auch bei einem Set prüft `in`, ob ein Wert enthalten ist

400 Absätze aus 217 verschiedenen Artikeln
 True

Liste

[0.2, 0.9, 0.4]

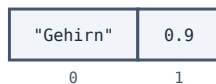


geordnet, veränderlich

Zugriff über die Position:
`xs[1]`

Tupel

("Gehirn", 0.9)

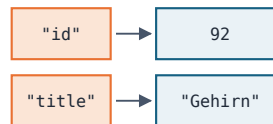


geordnet, unveränderlich

Zugriff über die Position:
`hit[0]`

Dict

{"id": 92, "title": "Gehirn"}

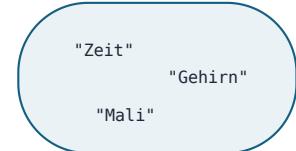


Schlüssel und Wert, veränderlich

Zugriff über den Schlüssel:
`p["title"]`

Set

{"Gehirn", "Zeit", "Mali"}



ohne Reihenfolge, ohne Duplikate

Prüfen, ob ein Wert vorkommt:
`"Zeit" in s`

Überblick: Vier Container

Liste, Tuple, Dict und Set sind vier häufig verwendete eingebaute Container von Python. Dieses Notebook nutzt von jedem nur das Nötigste, die volle Breite folgt in Grundkurs Teil 2.

Veränderlich heißt: Nach dem Anlegen lassen sich Elemente hinzufügen, ersetzen oder entfernen.

Container	Beispiel	Zugriff	veränderlich?	typischer Einsatz
Liste	[0.2, 0.9, 0.4]	über die Position: <code>xs[0]</code>	ja, zum Beispiel <code>xs.append(0.7)</code>	geordnete Folge von Werten, meist gleichartig. Beispiele: die Einträge eines Vektors, alle Absätze des Korpus
Tupel	("Gehirn", 0.9)	über die Position: <code>hit[0]</code> , oder durch Auspacken: <code>title, score = hit</code>	nein	festes Paket zusammengehöriger Werte. Beispiel: ein Treffer aus Titel und Ähnlichkeitswert
Dict	{"title": "Gehirn"}	über den Schlüssel: <code>p["title"]</code>	ja, zum Beispiel <code>p["lang"] = "de"</code>	benannte Felder und Zuordnungen. Beispiel: ein Absatz mit <code>id</code> , <code>title</code> und <code>text</code>
Set	{"Gehirn", "Zeit"}	kein Zugriff auf ein einzelnes Element. Möglich sind die Prüfung <code>"Zeit" in s</code> und das Durchlaufen mit <code>for</code>	ja, zum Beispiel <code>s.add("Mali")</code>	Duplikate entfernen und schnell prüfen, ob ein Wert vorkommt. Beispiel: die verschiedenen Titel des Korpus

Eine Liste darf Werte verschiedener Typen mischen. Üblich sind Listen mit gleichartigen Werten.

Damit lassen sich die Kursdaten vollständig lesen. Die Abbildung zeigt, wie sie zusammenhängen: Jede Anfrage nennt unter `expected` die Nummer des Absatzes, der gefunden werden soll. Absatz und Vektor stehen in `passages` und `vectors` jeweils unter demselben Index.

queries

8 Beispielanfragen, jede ein Dict

0	"text": "Wo liegt San Diego?" "expected": 108
1	"text": "Wer hat das Logo ...?" "expected": 164
⋮	
7	"text": "Was sind die ...?" "expected": 18

query_vectors

8 Listen mit je 384 Zahlen,

query_vectors[i] gehört zu queries[i]

passages

400 Absätze, jeder ein Dict

0	{"title": "Aufzugsanlage", ...}
1	{"title": "Sichuan", ...}
⋮	
108	"id": 108 "title": "San Diego" "text": "Geographie. San ..."
⋮	
399	{"title": "Mali", ...}

vectors[i] gehört zu passages[i]

vectors

400 Listen mit je 384 Zahlen

0	[-0.203, 0.192, ...]
1	[0.038, -0.221, ...]
⋮	
108	[0.242, -0.172, ...]
⋮	
399	[-0.195, 0.120, ...]

gleicher
Index
-----Nummer
des
Absatzes
→ **Aufgabe 2.2 #1**

5 min

Jede Beispielanfrage nennt im Feld `expected` die Nummer des Absatzes, der gefunden werden soll. Schreiben Sie die Funktion `expected_title(query_no)`. Sie gibt zu einer Anfragenummer (0 bis 7) den Titel dieses Absatzes zurück. Sehen Sie sich dazu zuerst `queries[0]` in einer eigenen Zelle an.

Wer früher fertig ist: Geben Sie in einer Schleife für alle acht Anfragen den Fragetext und den erwarteten Titel aus.

Hinweis: Es sind zwei Zugriffe nacheinander. Erst liefert `queries[query_no]["expected"]` die Nummer des Absatzes, dann führt diese Nummer in `passages` zum Absatz und dessen Titel.

```
def expected_title(query_no):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwelle – einfach ausführen
assert expected_title(0) == "San Diego", "Anfrage 0 fragt nach San Diego"
assert expected_title(3) == "Gehirn", "Anfrage 3 fragt nach dem Gehirn"
assert expected_title(7) == passages[queries[7]["expected"]]["title"]
print("Alle Prüfungen bestanden ✓")
```

2.3 Die Suche von Hand

M1 · Python-Grundlagen

 Wiederholung: Normalize, Compare, Select

Die Suche vergleicht den Vektor der Anfrage mit den Vektoren aller Absätze, in drei Schritten. **Normalize** bringt alle Vektoren auf die Länge 1. **Compare** berechnet das Skalarprodukt der Anfrage mit jedem Absatz. **Select** wählt die höchsten Werte aus. Eingeführt wurde das in LE 1, Abschnitt 1.1.1.

In der letzten Lerneinheit erledigte NumPy diese Rechnung in zwei Zeilen. Jetzt schreiben Sie jeden Baustein selbst, mit Listen, Schleifen und Funktionen. Für alle Funktionen dieses Kapitels gilt: Ein Vektor ist eine Liste von Zahlen. Zwei Vektoren, die miteinander verrechnet werden, haben gleich viele Einträge. Wo durch eine Länge geteilt wird, setzen wir Vektoren mit positiver Länge voraus. Den Nullvektor nehmen wir uns im Kapitel [2.6](#) vor.

2.3.1 Die Länge eines Vektors

Länge eines Vektors

Die Länge eines Vektors v mit n Einträgen ist die Wurzel aus der Summe der Quadrate seiner Einträge:

$$\|v\| = \sqrt{v_1^2 + v_2^2 + \dots + v_n^2}$$

Das ist der Satz des Pythagoras, verallgemeinert auf n Dimensionen. Der Vektor $(3, 4)$ hat die Länge 5, denn $9 + 16 = 25$. In Grundkurs Teil 1 haben Sie dieselbe Rechnung für zwei Dimensionen mit `math.sqrt` geschrieben.

Prüfzellen mit Fließkommazahlen. Rechnungen mit Fließkommazahlen liefern nicht immer exakt den erwarteten Wert, die letzte Nachkommastelle kann abweichen. Die Prüfzellen dieses Notebooks vergleichen solche Ergebnisse deshalb nicht mit `==`. Sie prüfen, ob der Abstand zum Sollwert sehr klein ist: `abs(actual - expected) < 1e-9`. Dabei liefert `abs` den Betrag einer Zahl, und `1e-9` ist die Schreibweise für 0,000000001. Woher die Abweichungen kommen, klärt das Kapitel [2.6](#).

Anschlusslösungen. Die Aufgaben dieses Notebooks bauen aufeinander auf. Unter den Prüfzellen steht deshalb jeweils ein Aufklappfeld „Anschlusslösung“ mit einer lauffähigen Fassung. Wenn Ihre eigene Lösung nach der Besprechung noch nicht läuft, kopieren Sie diese Fassung in Ihre Aufgabenzone und führen die Zelle aus. So können Sie ohne Lücke weiterarbeiten.

Das Wort „Länge“ hat ab jetzt zwei Bedeutungen. `len([3, 4])` liefert die Anzahl der Einträge, also 2. `norm([3, 4])` liefert die geometrische Länge, also 5. Beim Normalisieren ändert sich nur die geometrische Länge, die Anzahl der Einträge bleibt gleich.

Aufgabe 2.3.1 #1

5 min

Schreiben Sie die Funktion `norm(v)`. Sie gibt die Länge des Vektors `v` zurück. Die Eingabeliste bleibt unverändert.

Wer früher fertig ist: Prüfen Sie Ihre Funktion mit einem eigenen Beispiel. Wählen Sie dazu einen Vektor mit drei Einträgen, dessen Länge Sie von Hand ausrechnen können, zum Beispiel `[2.0, 3.0, 6.0]` ($4 + 9 + 36 = 49$, die Länge ist also 7). Schreiben Sie in eine neue Zelle eine `assert`-Zeile nach dem Muster der Prüfzelle.

Hinweis: Summieren Sie in einer Schleife die Quadrate aller Elemente in einer Variablen, die mit `0.0` beginnt. Die Wurzel liefert `math.sqrt`.

```
def norm(v):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfzelle – einfach ausführen
assert norm([3.0, 4.0]) == 5.0, "3, 4 und 5 bilden ein rechtwinkliges Dreieck"
assert norm([0.0, 0.0]) == 0.0, "der Nullvektor hat die Länge 0"
assert norm([-2.0]) == 2.0, "das Vorzeichen fällt durch das Quadrieren weg"
assert abs(norm([1.0, 1.0]) - 1.4142135623730951) < 1e-9, "Wurzel aus 2"
_v = [1.0, 2.0, 2.0]
assert norm(_v) == 3.0
assert _v == [1.0, 2.0, 2.0], "die Eingabeliste darf nicht verändert werden"
assert abs(norm(vectors[0]) - 2.980013441153241) < 1e-9, "Länge des ersten Absatzvektors"
print("Alle Prüfungen bestanden ✓")
```

2.3.2 Normalize

Normalisieren heißt, jeden Eintrag durch die Länge des Vektors zu teilen. Der Ergebnisvektor zeigt in dieselbe Richtung und hat die Länge 1. Ein Vektor der Länge 1 heißt Einheitsvektor. Die Funktion baut eine neue Liste auf und lässt die Eingabe unverändert. Das ist der Rumpf von `normalize_python` aus der letzten Lerneinheit, nur für einen einzelnen Vektor.

```
def normalize(v):
    length = norm(v)
    result = []
    for x in v:
        result.append(x / length)    # jeden Eintrag durch die Länge teilen
    return result                    # neue Liste, v bleibt unverändert

print(normalize([3.0, 4.0]))
print(norm(normalize([3.0, 4.0])))
```

```
[0.6, 0.8]
1.0
```

Die folgende Fassung stammt von einem KI-Assistenten. Sie liefert für `[3.0, 4.0]` dasselbe Ergebnis wie `normalize`. Die Anforderung lautet: Die Funktion gibt eine neue Liste zurück, die Eingabe bleibt unverändert.

Lernfragen 2.3.2 #1

1. Erfüllt `normalize_b` in der Zelle darunter diese Anforderung? Belegen Sie Ihr Urteil mit einem Test, den Sie in der Zelle ergänzen.

```
def normalize_b(v):
    length = norm(v)
    for i in range(len(v)):
        v[i] = v[i] / length
    return v

# Platz für Ihren Test:
```

2.3.3 Compare: das Skalarprodukt

Skalarprodukt und Kosinus-Ähnlichkeit

Das Skalarprodukt zweier Vektoren mit gleich vielen Einträgen multipliziert die Einträge paarweise und summiert die Produkte:

$$a \cdot b = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

Die Kosinus-Ähnlichkeit teilt das Skalarprodukt durch die Längen beider Vektoren:

$$\cos(\theta) = \frac{a \cdot b}{\|a\| \|b\|}$$

Haben beide Vektoren die Länge 1, ist der Nenner 1, und das Skalarprodukt ist bereits die Kosinus-Ähnlichkeit. Deshalb normalisiert die Suche zuerst und vergleicht dann nur noch mit dem Skalarprodukt.

Für das Skalarprodukt laufen zwei Listen im Gleichschritt. Hier ist die Schleife über den Index der passende Weg: `a[i]` und `b[i]` gehören zusammen.

Aufgabe 2.3.3 #1

5 min

Schreiben Sie die Funktion `dot(a, b)`. Sie gibt das Skalarprodukt zweier Vektoren mit gleich vielen Einträgen zurück. Schreiben Sie danach die Funktion `cosine(a, b)`, die daraus mit `norm` die Kosinus-Ähnlichkeit berechnet.

Wer früher fertig ist: Jeder Vektor zeigt in dieselbe Richtung wie er selbst. Überlegen Sie, welchen Wert `cosine(v, v)` deshalb liefern muss, und prüfen Sie es in einer neuen Zelle mit `print(cosine(vectors[0], vectors[0]))`.

Hinweis: `dot` ist wie `norm` aufgebaut, nur läuft die Schleife über `range(len(a))` und addiert `a[i] * b[i]`. `cosine` braucht keine eigene Schleife.

```
def dot(a, b):
    # TODO: Ihre Lösung hier
    ...

def cosine(a, b):
    # TODO: Ihre Lösung hier
    ...
```

```
# Prüfwerte – einfach ausführen
assert dot([1.0, 2.0, 3.0], [4.0, 5.0, 6.0]) == 32.0, "4 + 10 + 18"
assert dot([1.0, 0.0], [0.0, 1.0]) == 0.0, "senkrechte Vektoren haben das Skalarprodukt 0"
assert abs(cosine([2.0, 0.0], [5.0, 0.0]) - 1.0) < 1e-9, "gleiche Richtung ergibt 1"
assert abs(cosine([1.0, 0.0], [0.0, 3.0]) - 0.0) < 1e-9, "senkrecht ergibt 0"
assert abs(cosine([1.0, 2.0], [-1.0, -2.0]) + 1.0) < 1e-9, "entgegengesetzte Richtung ergibt -1"
# die Spielzeugvektoren aus der letzten Lerneinheit
dog = [0.70, -0.20, 0.50, 0.90]
cat = [0.55, -0.10, 0.65, 0.90]
car = [0.10, 0.70, -0.20, 0.30]
assert abs(cosine(dog, cat) - 0.9825573682516039) < 1e-9, "Hund und Katze: rund 0.98"
assert abs(cosine(dog, car) - 0.09991510822169683) < 1e-9, "Hund und Auto: rund 0.10"
print("Alle Prüfungen bestanden ✓")
```

Wenn zwei Listen im Gleichschritt durchlaufen werden, geht es in Python auch ohne Index. `zip` liefert die Elemente beider Listen paarweise. Beide Fassungen sind in Ordnung, mehr zu `zip` steht in Grundkurs Teil 2.

```
def dot_zip(a, b):
    total = 0.0
    ① for x, y in zip(a, b):
        total += x * y
    return total

print(dot([1.0, 2.0, 3.0], [4.0, 5.0, 6.0]), dot_zip([1.0, 2.0, 3.0], [4.0, 5.0, 6.0]))
```

① `zip(a, b)` liefert bei jedem Durchlauf ein Paar aus je einem Element beider Listen, das die Schleife gleich in `x` und `y` auspackt

```
32.0 32.0
```

2.3.4 Warum normalisieren?

Ein Experiment mit vier Vektoren in zwei Dimensionen. Die Anfrage hat den Vektor `q = [1, 1]`. Der Vektor von Dokument A zeigt exakt in dieselbe Richtung wie `q`. Der Vektor von Dokument B ist lang und zeigt um 45 Grad daneben. Der Vektor von Dokument C steht senkrecht auf `q`. Überlegen Sie vor dem Ausführen: Welches Dokument bekommt den höchsten Wert, wenn wir nur das Skalarprodukt berechnen, ohne zu normalisieren?

```
q = [1.0, 1.0]
docs = [("A", [1.0, 1.0]), ("B", [10.0, 0.0]), ("C", [0.5, -0.5])]
for name, d in docs:
    print(f"{name} Skalarprodukt: {dot(q, d):5.1f} Kosinus: {cosine(q, d):.2f}")
```

```
A Skalarprodukt: 2.0 Kosinus: 1.00
B Skalarprodukt: 10.0 Kosinus: 0.71
C Skalarprodukt: 0.0 Kosinus: 0.00
```

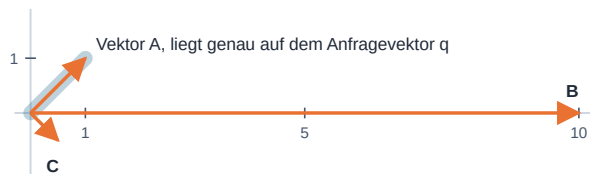
Richtung statt Länge

Beim rohen Skalarprodukt gewinnt Dokument B allein durch die Länge seines Vektors, obwohl der Vektor von A exakt in die Richtung des Anfragevektors zeigt. Mit der Kosinus-Ähnlichkeit zählt nur die Richtung, und A liegt vorn. Unsere Suche vergleicht die Richtungen der Embeddings, ihre Länge soll den Vergleich nicht beeinflussen. Deshalb gehört das Normalisieren dazu.

Andere Modelle und Suchverfahren können andere Ähnlichkeitsmaße verwenden, zum Beispiel das Skalarprodukt ohne Normalisierung oder den euklidischen Abstand.

Rohes Skalarprodukt

die Länge der Vektoren zählt mit



Skalarprodukt mit q

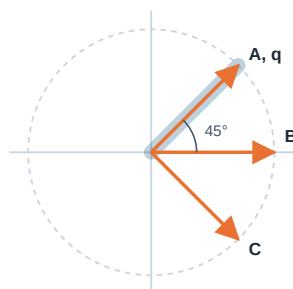
```
A 2.0
B 10.0 höchster Wert
C 0.0
```

die vier Vektoren

```
q = [1, 1]
A = [1, 1]
B = [10, 0]
C = [0.5, -0.5]
```

Nach dem Normalisieren

alle Vektoren haben die Länge 1, es zählt nur die Richtung



Kosinus mit q

```
A 1.00 höchster Wert
B 0.71
C 0.00
```

Legende

— Anfragevektor q

→ Vektoren der Dokumente A, B und C

--- Kreis mit Radius 1 (Länge 1)

Wie ist das bei den echten Daten? Die Absatzvektoren sind unterschiedlich lang, die Voraussetzung für denselben Effekt ist also gegeben. Wie stark er ausfällt, sehen wir, sobald die Suche steht.

```
lengths = []
for v in vectors:
    lengths.append(norm(v))
print(f"kürzester Absatzvektor: {min(lengths):.2f} längster: {max(lengths):.2f}")
```

```
kürzester Absatzvektor: 1.82 längster: 4.10
```

2.3.5 Select: alles zusammensetzen

Für den dritten Schritt haben Sie im letzten Lab schon alles geschrieben: `best_hit` und `top_hits` wählen aus einer Liste von Paaren die besten aus. Die nächste Zelle stellt beide Funktionen bereit, dazu `measure` zum Stoppen von Laufzeiten. Was fehlt, ist die Funktion, die in der letzten Lerneinheit vorgegeben war: `get_hits`.

```

# Vorgabe: Funktionen aus der letzten Lerneinheit – einfach ausführen
def best_hit(hits):
    best = hits[0]
    for title, score in hits:
        if score > best[1]:
            best = (title, score)
    return best

def top_hits(hits, k):
    remaining = list(hits)          # Arbeitskopie, das Original bleibt unberührt
    result = []
    for _ in range(min(k, len(remaining))):
        b = best_hit(remaining)
        result.append(b)
        remaining.remove(b)
    return result

def measure(f, *args, repeats=3):
    # f mehrfach ausführen und den Bestwert der gestoppten Zeiten zurückgeben
    times = []
    for _ in range(repeats):
        start = time.perf_counter()
        f(*args)
        times.append(time.perf_counter() - start)
    return min(times)

```

Aufgabe 2.3.5 #1

10 min

Schreiben Sie die Funktion `get_hits(query_no)`. Sie berechnet für die Beispielanfrage mit der Nummer `query_no` die Kosinus-Ähnlichkeit zu jedem der 400 Absätze und gibt eine Liste von Paaren `(title, score)` zurück, in der Reihenfolge der Absätze. Nutzen Sie `normalize` und `dot`. Die Kursdaten in `vectors` und `query_vectors` bleiben unverändert. Falls eine Fassung die Daten doch einmal verändert hat, führen Sie die Ladezelle am Anfang des Notebooks erneut aus, sonst rechnen alle folgenden Zellen mit den veränderten Werten.

Wer früher fertig ist: Die Prü fzelle testet die Anfragen 0 und 3. Schreiben Sie in eine neue Zelle einen eigenen Test für Anfrage 5. Er soll prüfen, dass der Treffer auf Platz 1 den Titel des erwarteten Absatzes trägt. Diesen Titel liefert Ihre Funktion `expected_title(5)`, den besten Treffer liefert `top_hits(get_hits(5), 1)`.

Hinweis: Normalisieren Sie zuerst den Anfragevektor. Laufen Sie dann mit `for i in range(len(passages))` über alle Absätze, denn Sie brauchen `passages[i]` und `vectors[i]` zusammen. Je Absatz entsteht ein Paar aus Titel und dem Skalarprodukt der beiden normalisierten Vektoren.

```

def get_hits(query_no):
    # TODO: Ihre Lösung hier
    ...

```

```

# Prüfwelle – einfach ausführen
assert abs(norm(vectors[0]) - 2.980013441153241) < 1e-9, "die Kursdaten sind verändert: bitte die Ladezelle
am Anfang erneut ausführen"
assert abs(norm(query_vectors[3]) - 5.294) < 0.01, "die Kursdaten sind verändert: bitte die Ladezelle am
Anfang erneut ausführen"
_vectors_before = [list(v) for v in vectors]                # unabhängige Kopien der Kursdaten
_query_vectors_before = [list(v) for v in query_vectors]
_hits = get_hits(0)
assert _hits is not None, "get_hits gibt noch nichts zurück (return vergessen?)"
assert len(_hits) == 400, "ein Paar je Absatz"
assert _hits[108][0] == "San Diego", "die Paare stehen in der Reihenfolge der Absätze"
assert abs(_hits[108][1] - 0.8309709895) < 1e-6, "Score des erwarteten Absatzes zu Anfrage 0"
assert top_hits(_hits, 1)[0][0] == "San Diego", "Anfrage 0 hat San Diego auf Platz 1"
_hits = get_hits(3)
assert top_hits(_hits, 1)[0][0] == "Gehirn", "Anfrage 3 hat das Gehirn auf Platz 1"
assert abs(_hits[92][1] - 0.8946394818) < 1e-6, "Score des erwarteten Absatzes zu Anfrage 3"
for _title, _score in _hits:
    assert -1.0 - 1e-9 <= _score <= 1.0 + 1e-9, "Kosinus-Werte liegen zwischen -1 und 1"
assert vectors == _vectors_before, "die Absatzvektoren dürfen nicht verändert werden (danach die Ladezelle
erneut ausführen)"
assert query_vectors == _query_vectors_before, "die Anfragevektoren dürfen nicht verändert werden (danach
die Ladezelle erneut ausführen)"
print("Alle Prüfungen bestanden ✓")

```

Damit steht die komplette Suche in reinem Python. Die Ausgabe für Anfrage 0 können Sie mit dem Appetizer der letzten Lerneinheit vergleichen, Treffer und Werte stimmen überein.

```

print("Anfrage:", queries[0]["text"])
for title, score in top_hits(get_hits(0), 3):
    print(f" {score:.3f} {title}")

```

```

Anfrage: Wo liegt San Diego?
0.831 San Diego
0.351 Mexiko-Stadt
0.338 North Carolina

```

Wie gut ist die Suche? Jede Beispielanfrage kennt den Absatz, der gefunden werden soll. Die Daten sind also gelabelt: Zu jeder Anfrage ist das erwartete Ergebnis bekannt. Die nächste Zelle zählt, bei wie vielen der acht Anfragen dieser Absatz auf Platz 1 landet. Verglichen wird die Nummer des Absatzes, nicht der Titel: Mehrere Absätze können denselben Titel tragen, die Nummer ist eindeutig.

```

correct = 0
for query_no in range(len(queries)):
    hits = get_hits(query_no)
    best_no = 0                # Nummer des bisher besten Absatzes
    for i in range(len(hits)):
        if hits[i][1] > hits[best_no][1]:
            best_no = i
    if best_no == queries[query_no]["expected"]:
        correct += 1
print(correct, "von", len(queries), "Beispielanfragen finden den erwarteten Absatz auf Platz 1")

```

```

8 von 8 Beispielanfragen finden den erwarteten Absatz auf Platz 1

```

Das Ergebnis gilt für diese acht vorbereiteten Anfragen. Sie wurden passend zum Korpus ausgewählt und sagen wenig darüber, wie die Suche bei beliebigen Fragen abschneidet.

Jetzt lässt sich auch die offene Frage klären, was das Normalisieren bei den echten Daten ausmacht. Die nächste Zelle berechnet die Rangfolge für Anfrage 0 einmal mit normalisierten Vektoren und einmal mit dem rohen Skalarprodukt.

```

raw_hits = []
for i in range(len(passages)):
    raw_hits.append((passages[i]["title"], dot(query_vectors[0], vectors[i])))

def titles_of(hits):
    # nur die Titel einer Trefferliste
    result = []
    for title, score in hits:
        result.append(title)
    return result

print("mit Normalisieren: ", titles_of(top_hits(get_hits(0), 4)))
print("ohne Normalisieren: ", titles_of(top_hits(raw_hits, 4)))

```

```

mit Normalisieren:  ['San Diego', 'Mexiko-Stadt', 'North Carolina', 'Florida']
ohne Normalisieren: ['San Diego', 'Florida', 'North Carolina', 'Mexiko-Stadt']

```

Platz 1 bleibt gleich, weil der richtige Absatz mit großem Abstand vorn liegt. Dahinter verschiebt sich die Reihenfolge: Ohne Normalisieren rückt ein langer Vektor nach vorn. Bei den acht Beispielanfragen ändert sich Platz 1 in keinem Fall, die Plätze dahinter ändern sich in jedem.

Zum Schluss dieses Kapitels messen wir, wie lange eine Anfrage dauert. Den Wert brauchen wir gleich wieder.

```

t_get_hits = measure(get_hits, 0)
print(f"eine Anfrage mit get_hits: {t_get_hits * 1000:.1f} ms")

```

```

eine Anfrage mit get_hits: 11.9 ms

```

2.4 Klassen: die Suche als Objekt

M1 · Python-Grundlagen

Kurz überlegen

`get_hits` rechnet bei jeder Anfrage alles neu. Welcher Teil dieser Rechnung hängt von der Anfrage ab, und welcher nicht?

Die Antwort führt zu einer Vorberechnung: Wir normalisieren die Absatzvektoren einmal und verwenden das Ergebnis bei jeder Suche wieder. Dafür müssen die vorbereiteten Daten und die Funktionen, die mit ihnen arbeiten, zusammengehalten werden. Das kann eine Klasse leisten. Nötig ist sie dafür nicht: Die normalisierten Vektoren ließen sich auch in einer weiteren Variablen ablegen und an eine Funktion übergeben. Die Klasse hält die vorbereiteten Daten und die Funktionen, die sich auf sie verlassen, an einer Stelle zusammen. Klassen kennen Sie aus C++, neu ist im Wesentlichen die Schreibweise.

get_hits(query_no)

rechnet bei jeder Anfrage alles neu

**Corpus** erledigt die Vorarbeit einmal und bewahrt das Ergebnis auf

Legende Schritt, der bei jeder Anfrage läuft Schritt, der einmal läuft → Reihenfolge - - -> gespeichertes Ergebnis wird gelesen

```

1 class Corpus:
2     def __init__(self, passages, vectors):
3         self.passages = passages
4         self.vectors = []
5         for v in vectors:
6             self.vectors.append(normalize(v))    # einmal normalisieren und aufbewahren
7
8     def scores(self, query_vector):
9         q = normalize(query_vector)
10        hits = []
11        for i in range(len(self.passages)):
12            hits.append((self.passages[i]["title"], dot(q, self.vectors[i])))
13        return hits
14
15     def search(self, query_vector, k=3):
16        return top_hits(self.scores(query_vector), k)
  
```

- ① `class` leitet die Definition ein, der Rumpf ist eingerückt wie bei einer Funktion
- ② `__init__` wird beim Anlegen eines Objekts aufgerufen und übernimmt die Rolle des Konstruktors; `self` ist das Objekt selbst und steht in jeder Methode als erster Parameter
- ③ eine Zuweisung an `self.name` legt ein Attribut des Objekts an, eine Deklaration gibt es nicht
- ④ eine Methode ist eine Funktion im Rumpf der Klasse; über `self` erreicht sie die Attribute
- ⑤ Methoden desselben Objekts werden ebenfalls über `self` aufgerufen

Ein Objekt, also eine Instanz der Klasse, entsteht durch den Aufruf des Klassennamens. Python ruft dabei `__init__` auf, das `self` wird nicht mit übergeben. Methoden werden mit einem Punkt am Objekt aufgerufen.

```

corpus = Corpus(passages, vectors)    # legt das Objekt an und normalisiert alle Absatzvektoren
for title, score in corpus.search(query_vectors[0]):
    print(f" {score:.3f} {title}")
  
```

```

0.831 San Diego
0.351 Mexiko-Stadt
0.338 North Carolina
  
```

Im Direktvergleich zeigt eine kleine Zählerklasse, was gleich bleibt und was sich in der Schreibweise ändert.

C++

```
class Counter {
public:
    int count;

    Counter(int start) {
        count = start;
    }

    void add(int n) {
        count += n;
    }
};

Counter c(10);
c.add(5);
```

Python

```
class Counter:

    def __init__(self, start):
        self.count = start

    def add(self, n):
        self.count += n

c = Counter(10)
c.add(5)
```

Klassen in Python und C++

Das Konzept ist dasselbe, die Schreibweise unterscheidet sich an vier Stellen.

	C++	Python
Konstruktor	trägt den Namen der Klasse	heißt <code>__init__</code>
das Objekt selbst	<code>this</code> , steht in jeder Methode bereit, ohne in der Parameterliste zu stehen	<code>self</code> , steht ausdrücklich als erster Parameter in jeder Methode
Attribute	werden in der Klasse deklariert	entstehen durch Zuweisung an <code>self.name</code> , meist in <code>__init__</code>
Sichtbarkeit	<code>public</code> , <code>private</code> , <code>protected</code> , der Compiler prüft sie	keine Schlüsselwörter. Ein führender Unterstrich (<code>_name</code>) bedeutet nach Konvention „intern“. Der Interpreter prüft das nicht, auch ein solches Attribut lässt sich von außen lesen und ändern

Vertiefung: Warum steht self in der Parameterliste?

NICHT PRÜFUNGSRELEVANT

Eine Methode ist in Python eine gewöhnliche Funktion, die im Rumpf der Klasse steht. Der Aufruf `corpus.search(q)` ist eine Kurzschreibweise für `Corpus.search(corpus, q)`. Python übergibt das Objekt als erstes Argument, und der Parameter dafür heißt nach Konvention `self`.

Auch C++ übergibt das Objekt an die Methode, dort geschieht es verdeckt über den Zeiger `this`. Python macht die Übergabe sichtbar. Beide Schreibweisen liefern dasselbe Ergebnis:

```
hits_short = corpus.search(query_vectors[0], 1)      # die übliche Schreibweise
hits_long = Corpus.search(corpus, query_vectors[0], 1) # das Objekt als erstes Argument
print(hits_short == hits_long)                      # True
```

Zwei typische Fehler. Beide haben mit `self` zu tun, und beide erkennen Sie an ihrer Meldung.

Fehlt `self` in der Parameterliste einer Methode, passt die Zahl der Argumente nicht, denn Python übergibt das Objekt immer mit:

```
class Corpus:
    def size():
        return 400

Corpus().size()
# TypeError: Corpus.size() takes 0 positional arguments but 1 was given
```

Fehlt `self`. bei der Zuweisung, entsteht nur eine lokale Variable, die am Ende von `__init__` verschwindet. Der Fehler zeigt sich erst beim späteren Zugriff:

```
class Corpus:
    def __init__(self, passages):
        passages = passages          # self. fehlt
```

```
Corpus(passages).passages
# AttributeError: 'Corpus' object has no attribute 'passages'
```

Was bringt die Vorberechnung? Die nächste Zelle misst das Anlegen des Objekts und eine einzelne Suche getrennt und stellt beides neben `get_hits`. `corpus.scores` liefert dieselbe Liste von Paaren wie `get_hits`, die Messungen sind also vergleichbar.

```
t_build = measure(Corpus, passages, vectors)          # einmalig: alle Absatzvektoren normalisieren
t_scores = measure(corpus.scores, query_vectors[0])    # je Anfrage: nur noch die Skalarprodukte
print(f"Objekt anlegen (einmalig):  {t_build * 1000:5.1f} ms")
print(f"eine Anfrage mit dem Objekt: {t_scores * 1000:5.1f} ms")
print(f"eine Anfrage mit get_hits:   {t_get_hits * 1000:5.1f} ms")
```

```
Objekt anlegen (einmalig):      8.1 ms
eine Anfrage mit dem Objekt:   4.4 ms
eine Anfrage mit get_hits:     11.9 ms
```

Der Gewinn kommt aus der Vorberechnung

Schneller wird die Suche nicht durch die Klasse, sondern weil die Absatzvektoren nur noch einmal normalisiert werden. Die Klasse sorgt dafür, dass die vorbereiteten Daten und die Methoden, die sich auf sie verlassen, zusammenbleiben. Die Vorarbeit kostet einmalig Zeit und zahlt sich mit jeder weiteren Anfrage aus.



Was für jede Bestellung gleich ist, bereitet die Küche einmal vor und kocht nicht jede einzelne Bestellung „from scratch“. Genauso normalisiert `Corpus` die Absatzvektoren einmal und verwendet sie bei jeder Anfrage wieder.

Sondermethoden. Für eine Liste funktionieren `len(xs)` und `xs[0]`. Für ein `Corpus`-Objekt funktionieren sie bisher nicht: `len(corpus)` endet mit `TypeError: object of type 'Corpus' has no len()`, und `corpus[0]` endet mit `TypeError: 'Corpus' object is not subscriptable`. Python sucht für diese Schreibweisen im Objekt nach Methoden mit festgelegten Namen. Die Namen beginnen und enden mit zwei Unterstrichen.

Schreibweise	Python ruft auf	Ergebnis
<code>len(corpus)</code>	<code>corpus.__len__()</code>	die Anzahl der Elemente

Schreibweise	Python ruft auf	Ergebnis
<code>corpus[0]</code>	<code>corpus.__getitem__(0)</code>	das Element an dieser Position
<code>repr(corpus)</code>	<code>corpus.__repr__()</code>	die Darstellung als Text, die auch das Notebook für ein Objekt anzeigt

In C++ heißt dieses Prinzip Operatorüberladung.

Aufgabe 2.4 #1

10 min

Die Zelle darunter enthält die Klasse noch einmal, ergänzt um `__repr__`. Schreiben Sie `__len__` (Anzahl der Absätze) und `__getitem__` (der Absatz an der Position `i` in `self.passages`, also das Dict mit `id`, `title` und `text`). Die Position ist unabhängig vom Feld `id` des Absatzes. Führen Sie die Zelle danach vollständig aus: Die letzte Zeile legt `corpus` neu an. Ein Objekt, das vor der Änderung angelegt wurde, gehört noch zur alten Fassung der Klasse und kennt die neuen Methoden nicht.

Wer früher fertig ist: Probieren Sie in einer neuen Zelle `corpus[-1]` aus. Erklären Sie, warum der negative Index funktioniert, obwohl Ihre Methode ihn nicht eigens behandelt.

Hinweis: Beide Methoden sind Einzeiler, die an `self.passages` weiterreichen, was sie gefragt werden.

```
class Corpus:
    def __init__(self, passages, vectors):
        self.passages = passages
        self.vectors = []
        for v in vectors:
            self.vectors.append(normalize(v))    # einmal normalisieren und aufbewahren

    def scores(self, query_vector):
        q = normalize(query_vector)
        hits = []
        for i in range(len(self.passages)):
            hits.append((self.passages[i]["title"], dot(q, self.vectors[i])))
        return hits

    def search(self, query_vector, k=3):
        return top_hits(self.scores(query_vector), k)

    def __repr__(self):
        return f"Corpus({len(self.passages)} Absätze, {len(self.vectors[0])} Dimensionen)"

    def __len__(self):
        # TODO: Ihre Lösung hier
        ...

    def __getitem__(self, i):
        # TODO: Ihre Lösung hier
        ...

corpus = Corpus(passages, vectors)    # neu anlegen, damit das Objekt zur neuen Fassung der Klasse gehört
```

```
# Prüfwelle – einfach ausführen (legt ihr eigenes Objekt an)
_c = Corpus(passages, vectors)
assert len(_c) == 400, "__len__ liefert die Anzahl der Absätze"
assert len(Corpus(passages[:3], vectors[:3])) == 3, "die Anzahl darf nicht fest eingetragen sein"
assert type(_c[0]) == dict, "__getitem__ liefert den ganzen Absatz, also das Dict"
assert _c[0]["title"] == "Aufzugsanlage", "der erste Absatz"
assert _c[399] == passages[399]
assert _c.search(query_vectors[0], 1)[0][0] == "San Diego", "die Suche funktioniert weiterhin"
# ein zweites Objekt mit eigenen Daten: Die Methoden müssen mit den Daten des Objekts arbeiten
_probe = Corpus([{"id": 999, "title": "Testabsatz", "text": "Test"}], [[3.0, 4.0]])
assert len(_probe) == 1, "__len__ muss die Absätze des Objekts zählen (self.passages)"
assert _probe[0]["title"] == "Testabsatz", "__getitem__ muss auf self.passages zugreifen, nicht auf passages"
print("Alle Prüfungen bestanden ✓")
```

Mit den drei Sondermethoden verhält sich das Objekt an diesen Stellen wie eine Liste.

```
print(repr(corpus))
print(len(corpus))
print(corpus[0]["title"])
```

```
Corpus(400 Absätze, 384 Dimensionen)
400
Aufzugsanlage
```

Vertiefung: Warum auch die for-Schleife funktioniert

NICHT PRÜFUNGSRELEVANT

Mit `__getitem__` lässt sich das Objekt ohne weiteres Zutun durchlaufen: `for p in corpus:` funktioniert. Python fragt dafür der Reihe nach `corpus[0]`, `corpus[1]`, `corpus[2]` und so weiter ab und beendet die Schleife, sobald ein Zugriff mit einem `IndexError` scheitert. Diesen Fehler löst hier die Liste `self.passages` aus, an die `__getitem__` weiterreicht. Eine `__getitem__`-Methode, die bei zu großen Nummern keinen `IndexError` auslöst, würde die Schleife nie beenden. Der übliche Weg, ein Objekt durchlaufbar zu machen, ist die Sondermethode `__iter__`.

Die Schnittstelle aus `__len__` und `__getitem__` begegnet Ihnen später im Semester wieder. Wir verwenden sie dann für PyTorch-Datensätze, auf die über eine Nummer zugegriffen wird: `__getitem__` liefert ein Beispiel, `__len__` die Anzahl der Beispiele.

Aufgabe 2.4 #2

5 min

Die Klasse `Corpus` arbeitet mit beliebigen Daten, nicht nur mit den 400 Absätzen. Die Vorgabe-Zelle darunter stellt drei neue Absätze mit zweidimensionalen Vektoren und einen Anfragevektor bereit. Lesen Sie die Vektoren als Pfeile in der Ebene: Der erste Eintrag zeigt nach Osten, der zweite nach Norden.

1. Sagen Sie vorher, welcher Absatz bei der Suche auf Platz 1 liegt, und notieren Sie Ihre Vorhersage als Kommentar.
2. Legen Sie mit den neuen Daten ein Objekt `mini` an und speichern Sie den Titel des besten Treffers in `best_title`.
3. Schreiben Sie einen eigenen Test mit `assert`, der fehlschlägt, wenn das Normalisieren der Absatzvektoren fehlt. Begründen Sie in einem Satz als Kommentar, warum Ihr Test diesen Fehler aufdeckt.

Wer früher fertig ist: Berechnen Sie in einer neuen Zelle für jeden der drei Absätze das rohe Skalarprodukt mit dem Anfragevektor, also `dot(mini_query, v)`. Welcher Absatz läge ohne Normalisieren auf Platz 1?

Hinweis: Das Objekt entsteht wie `corpus`, nur mit den neuen Listen. `search` liefert eine Liste von Paaren `(title, score)`, der Titel des besten Treffers steht also an der Stelle `[0][0]`. Für den Test: Das rohe Skalarprodukt belohnt lange Vektoren.

```
# Vorgabe: drei neue Absätze, ihre Vektoren und ein Anfragevektor – einfach ausführen
mini_passages = [{"id": 0, "title": "Nord", "text": "Absatz Nord"},
                  {"id": 1, "title": "Ost", "text": "Absatz Ost"},
                  {"id": 2, "title": "Nordost", "text": "Absatz Nordost"}]
mini_vectors = [[0.0, 2.0], [9.0, 1.0], [1.0, 1.0]]      # je Absatz ein Vektor: [nach Osten, nach Norden]
mini_query = [2.0, 3.0]                                # der Anfragevektor
```

```
# 1. Vorhersage: Welcher Absatz liegt auf Platz 1?
# TODO: Ihre Lösung hier
...

# 2. Objekt anlegen, Titel des besten Treffers in best_title speichern
# TODO: Ihre Lösung hier
...

# 3. eigener Test mit assert, darunter die Begründung als Kommentar
# TODO: Ihre Lösung hier
...
```

```
# Prüfwerte – einfach ausführen
assert type(mini).__name__ == "Corpus", "mini ist ein Objekt der Klasse Corpus"
assert len(mini) == 3, "mini enthält die drei neuen Absätze"
assert mini[2]["title"] == "Nordost", "mini arbeitet mit mini_passages"
assert best_title == mini.search(mini_query, 1)[0][0], "best_title ist der Titel des besten Treffers"
for _title, _score in mini.scores(mini_query):
    assert -1.0 - 1e-9 <= _score <= 1.0 + 1e-9, "Kosinus-Werte liegen zwischen -1 und 1: Fehlt das Normalisieren?"
assert len(corpus) == 400, "corpus bleibt das Objekt mit den 400 Absätzen"
print("Alle Prüfungen bestanden ✓")
```

Lernfragen 2.4 #2

1. Die Klasse `Corpus` bewahrt die normalisierten Absatzvektoren auf. Was muss neu berechnet werden, wenn nur die Anfrage wechselt? Was muss neu berechnet werden, wenn sich ein Absatzvektor ändert?

2.5 Was kostet Python?

2.5.1 Erst schätzen, dann messen

Erst schätzen, dann messen

Die Suche steht jetzt zweimal zur Verfügung: in reinem Python und als NumPy-Fassung aus der letzten Lerneinheit. Um welchen Faktor, schätzen Sie, ist NumPy bei einer Anfrage über 400 Absätze schneller: 5, 50 oder 500? Legen Sie sich fest, bevor Sie die nächste Zelle ausführen.

Damit der Vergleich fair ist, erledigen beide Funktionen dieselbe Arbeit: Sie normalisieren die Anfrage und alle 400 Absatzvektoren und berechnen die 400 Ähnlichkeitswerte. Ausgaben und das Laden der Daten bleiben außerhalb der Messung. Ein Unterschied bleibt und gehört dazugesagt: Die Python-Fassung rechnet mit Python-Zahlen (64 Bit), NumPy mit den gespeicherten 32-Bit-Zahlen. Die NumPy-Zeilen müssen Sie heute nicht im Detail verstehen, sie sind das Thema der nächsten Lerneinheit.

```

# Vorgabe: beide Fassungen berechnen die 400 Ähnlichkeitswerte zu einer Anfrage
def scores_python(query_no):
    q = normalize(query_vectors[query_no])
    result = []
    for v in vectors:
        result.append(dot(q, normalize(v)))
    return result

E = np.load("data/appetizer_embeddings.npy")          # die Absatzvektoren als NumPy-Array
Q = np.load("data/appetizer_query_embeddings.npy")    # die Anfragevektoren als NumPy-Array

def scores_numpy(query_no):
    q = Q[query_no] / np.linalg.norm(Q[query_no])
    return (E / np.linalg.norm(E, axis=1, keepdims=True)) @ q

t_python = measure(scores_python, 0)
t_numpy = measure(scores_numpy, 0, repeats=20)
print(f"reines Python: {t_python * 1000:7.2f} ms")
print(f"NumPy:         {t_numpy * 1000:7.2f} ms")
print(f"Faktor:        {t_python / t_numpy:7.0f}")

```

```

reines Python:    12.60 ms
NumPy:            0.12 ms
Faktor:           104

```

Die Zahlen im Skript sind auf einem Referenzrechner gemessen, Ihre Werte weichen ab. Die Größenordnung bleibt. Die Mathematik ist in beiden Fassungen dieselbe. Woher kommt der Unterschied? Das untersuchen wir im nächsten Abschnitt.

2.5.2 Das Laufzeitmikroskop

Wir sehen uns an, was im Speicher liegt. Alle Angaben in diesem Abschnitt gelten für CPython, die übliche Python-Implementierung, auf einem 64-Bit-System. So laufen Colab und die Kursumgebung. In C++ belegt auf diesen Plattformen ein `double` 8 Bytes und ein `float` 4 Bytes, und ein Array legt die Werte direkt hintereinander ab. In Python ist jede Zahl ein eigenes Objekt. `sys.getsizeof` zeigt, wie viel Speicher ein Objekt belegt. Als Beispiel dient der Vektor des ersten Absatzes aus den Kursdaten, eine Liste mit 384 Zahlen.

```

x = 1.0
print(sys.getsizeof(x), "Bytes belegt eine einzelne Python-Fließkommazahl")
v = vectors[0]                                # der Vektor des ersten Absatzes: eine Liste mit 384 Zahlen
print(sys.getsizeof(v), "Bytes belegt diese Liste, ohne die Zahlen darin")

```

```

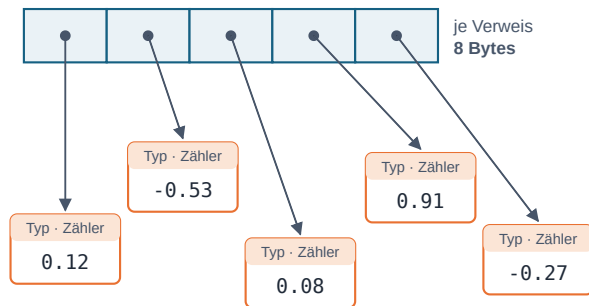
24 Bytes belegt eine einzelne Python-Fließkommazahl
3128 Bytes belegt diese Liste, ohne die Zahlen darin

```

Das Objekt einer Fließkommazahl enthält die 8 Bytes des eigentlichen Zahlenwerts und dazu 16 Bytes Verwaltungsdaten: den Typ und einen Zähler, der festhält, wie viele Namen und Container auf das Objekt verweisen. Die Liste selbst speichert keine Zahlen. Sie enthält je Element einen Verweis von 8 Bytes auf ein Zahlenobjekt, das irgendwo im Speicher liegt.

Python-Liste

ein Feld von Verweisen, jede Zahl ist ein eigenes Objekt



je Objekt 24 Bytes: 8 Bytes Zahlenwert und 16 Bytes Verwaltung

Zusammenhängender Block

alle Zahlen gleichartig und direkt hintereinander

so liegen die Werte in einem C-Array und in NumPy

0.12	-0.53	0.08	0.91	-0.27
------	-------	------	------	-------

je Zahl 4 Bytes (float32), keine Verweise, keine Verwaltung je Zahl

Eine Zeile wie `total += x * x` kostet in der Python-Schleife bei jedem Durchlauf mehr als die beiden Rechenoperationen. In C++ stehen die Typen beim Übersetzen fest, der Compiler legt die passenden Maschinenbefehle einmal vorab fest. Multipliziert und addiert wird auch dort in jedem Durchlauf. Es entfallen aber die Typprüfungen und die Verwaltung von Python-Zahlenobjekten.

In Python sieht ein Durchlauf vereinfacht so aus: Der Interpreter folgt dem Verweis aus der Liste zum Zahlenobjekt. Er prüft den Typ des Objekts, denn eine Liste kann Objekte beliebiger Typen enthalten. Er wählt die Multiplikation aus, die zu diesem Typ gehört, und führt sie aus. Für das Ergebnis legt er ein neues Zahlenobjekt an. Dasselbe wiederholt sich für die Addition. Die eigentliche Rechnung ist der kleinste Teil dieser Arbeit. Diese Beschreibung ist vereinfacht. Aktuelle CPython-Versionen merken sich bei häufig ausgeführten Stellen die passende Operation und sparen einen Teil der Auswahl. Die Objekte und die Verweise bleiben.



Wie ein Prüfer, der jedes Formular erst an die Schablone hält, erledigt der Interpreter vor jeder einzelnen Rechnung die Formalitäten: Verweis verfolgen, Typ prüfen, Operation auswählen. Die Rechnung selbst ist der kleinste Teil der Arbeit.

Hilft kompakter Speicher? Die Standardbibliothek bringt mit `array.array` einen Container mit, der Zahlen so ablegt wie ein C-Array: gleichartig und direkt hintereinander, hier mit 4 Bytes je Zahl.

```
import array
① a = array.array("f", vectors[0])      # der Vektor des ersten Absatzes, dicht gepackt
② print(a.itemsize, "Bytes je Zahl,", a.itemsize * len(a), "Bytes für alle 384 Zahlen")
```

① `array.array("f", ...)` legt die Zahlen dicht gepackt ab, das "f" steht für Fließkommazahlen mit 32 Bit

② `itemsize` ist der Speicherbedarf je Zahl in Bytes

4 Bytes je Zahl, 1536 Bytes für alle 384 Zahlen

Erst schätzen, dann messen

Die nächste Zelle berechnet die geometrische Länge eines Vektors mit einer Million Einträgen dreimal: über eine Python-Liste, über ein `array.array` und mit NumPy. Die ersten beiden nutzen Ihre Funktion `norm`. Ist die Schleife über das kompakte `array.array` schneller als die über die Liste?

```
big_list = []
for i in range(1_000_000):
    big_list.append(i * 0.5)           # eine Million Zahlen als Liste
big_array = array.array("f", big_list) # dieselben Zahlen dicht gepackt
big_numpy = np.array(big_list, dtype=np.float32) # dieselben Zahlen als NumPy-Array

t_loop_list = measure(norm, big_list)
t_loop_array = measure(norm, big_array)
t_loop_numpy = measure(np.linalg.norm, big_numpy)
print(f"Schleife über die Liste:      {t_loop_list * 1000:7.2f} ms")
print(f"Schleife über das array:      {t_loop_array * 1000:7.2f} ms")
print(f"NumPy:                       {t_loop_numpy * 1000:7.2f} ms")
```

```
Schleife über die Liste:      18.05 ms
Schleife über das array:      20.51 ms
NumPy:                       0.14 ms
```

Die Schleife über das `array.array` ist nicht schneller, meist sogar etwas langsamer. Der Grund: Die Schleife läuft weiter im Interpreter und rechnet mit Zahlenobjekten. Bei jedem Zugriff muss Python aus den rohen 4 Bytes erst ein solches Objekt bauen. Die Liste hat die fertigen Objekte schon.

Kompakter Speicher allein genügt nicht

Schnell wird die Rechnung erst, wenn beides zusammenkommt: Die Zahlen liegen als zusammenhängender Block im Speicher, und die Schleife über diesen Block läuft in Maschinencode statt im Interpreter. Genau das leistet NumPy, und damit beginnt die nächste Lerneinheit.

Aufgabe 2.5.2 #1

5 min

Schätzen Sie den Speicherbedarf der Kursvektoren, 400 Vektoren mit je 384 Zahlen. Berechnen Sie `bytes_lists` für die Ablage als Python-Listen: je Zahl ein Objekt und ein Verweis. Berechnen Sie `bytes_block` für die Ablage als Block mit 4 Bytes je Zahl. Die Größen aus diesem Abschnitt genügen dafür.

Wer früher fertig ist: Die Rechnung berücksichtigt nur die Zahlenobjekte und die Verweise auf sie. Die 401 Listen belegen zusätzlich je 56 Bytes für ihre eigene Verwaltung, und die äußere Liste hält 400 Verweise auf die inneren Listen. Rechnen Sie aus, wie viele Bytes dadurch hinzukommen.

Hinweis: Rechnen Sie mit den Größen unserer Umgebung: Ein Zahlenobjekt belegt 24 Bytes, ein Verweis in der Liste 8 Bytes.

```
n = 400 * 384                                # Anzahl der Zahlen in den Kursvektoren
# TODO: Ihre Lösung hier
...
```

```
# Prüfwerte – einfach ausführen
assert bytes_lists == 4_915_200, "je Zahl 24 Bytes für das Objekt und 8 Bytes für den Verweis"
assert bytes_block == 614_400, "je Zahl 4 Bytes"
print(f"als Listen: {bytes_lists / 1e6:.2f} MB   als Block: {bytes_block / 1e6:.2f} MB   Faktor {bytes_lists / bytes_block:.0f}")
print("Alle Prüfungen bestanden ✓")
```

Zum Nachmessen muss `sys.getsizeof` auf jeder Ebene einzeln aufgerufen werden: Für die äußere Liste liefert die Funktion nur deren eigene Größe, weder die inneren Listen noch die Zahlenobjekte zählen mit.

```

measured = sys.getsizeof(vectors)           # die äußere Liste
for v in vectors:
    measured += sys.getsizeof(v)             # jede innere Liste
    for x in v:
        measured += sys.getsizeof(x)        # jedes Zahlenobjekt
print(f"gemessen: {measured / 1e6:.2f} MB")

```

gemessen: 4.94 MB

2.5.3 Was kostet ein Container?

Auch die Wahl des Containers kostet oder spart Zeit. Ein häufiger Fall ist die Frage, ob ein Wert in einem Container vorkommt. Schätzen Sie zuerst: Eine Liste und ein Set enthalten dieselbe Million Zahlen. Gesucht wird ein Wert, der nicht vorkommt. Um welchen Faktor unterscheiden sich die Suchzeiten?

Einzelne Suchen im Set sind zu kurz für eine verlässliche Messung. Die Zelle misst deshalb viele Suchen am Stück und teilt die Zeit durch deren Anzahl. Beide Container sind vor der Messung fertig aufgebaut.

```

def search_many(container, value, repeats):
    for _ in range(repeats):
        value in container                # das Ergebnis wird nicht gebraucht, es geht um die Zeit

print(f"{'Elemente':>10s}{{'Liste':>14s}{{'Set':>12s}}")
for n in [1_000, 10_000, 100_000, 1_000_000]:
    xs = list(range(n))                  # Liste mit den Zahlen 0 bis n-1
    s = set(xs)                          # dieselben Zahlen als Set
    t_list = measure(search_many, xs, -1, 10) / 10          # Zeit je Suche in der Liste
    t_set = measure(search_many, s, -1, 100_000) / 100_000  # Zeit je Suche im Set
    print(f"{n:>10,}{t_list * 1e6:>11.1f} µs {t_set * 1e6:>9.2f} µs")

```

Elemente	Liste	Set
1,000	4.2 µs	0.02 µs
10,000	39.8 µs	0.02 µs
100,000	428.9 µs	0.02 µs
1,000,000	4288.5 µs	0.02 µs

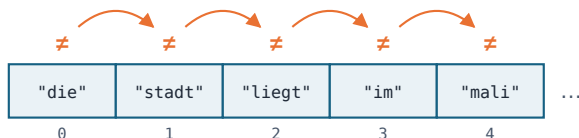
Die Liste wird Element für Element durchsucht. Fehlt der Wert, muss Python alle Elemente ansehen, und die Zeit wächst mit der Länge der Liste. Das Set berechnet aus dem gesuchten Wert eine Kennzahl, den Hashwert, und sieht nur an der Stelle nach, die zu dieser Kennzahl gehört. Die Suchzeit hängt dadurch im Mittel kaum von der Größe des Sets ab.

„Im Mittel“ ist dabei wörtlich gemeint. Verschiedene Werte können auf dieselbe Stelle fallen, dann sind einige zusätzliche Vergleiche nötig. Python hält die Tabelle so groß, dass das selten vorkommt. Ein Dict arbeitet mit seinen Schlüsseln auf dieselbe Weise, deshalb ist auch der Zugriff `d[key]` unabhängig von der Größe des Dicts schnell. Der Preis ist Speicher: Set und Dict belegen mehr Platz als eine Liste mit denselben Elementen.

Liste

wird Element für Element durchsucht

"python" in words

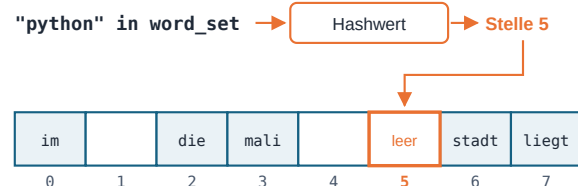


Der Wert fehlt: Python vergleicht mit allen Elementen.

Die Zeit wächst mit der Länge der Liste.

Set

der Hashwert führt direkt zur richtigen Stelle



Die Stelle ist leer: Der Wert fehlt, ein Blick genügt.

Die Zeit hängt im Mittel kaum von der Größe ab.



Mit der Marke führt der Weg direkt zum richtigen Haken, so wie der Hashwert im Set direkt zur richtigen Stelle führt. Ohne Marke bleibt nur, alle Mäntel der Reihe nach durchzusehen, wie in der Liste.

Vertiefung: Wie sind Set und Dict gespeichert?

NICHT PRÜFUNGSRELEVANT

Beide nutzen intern eine Hashtabelle. Das ist ein Feld mit festen Plätzen, und der Hashwert bestimmt, an welchem Platz ein Element liegt. Ein Teil der Plätze bleibt frei. Deshalb belegen Set und Dict ein Mehrfaches einer Liste mit denselben Elementen.

Jeder belegte Platz hält den Hashwert und einen Verweis auf das Objekt. Beim Dict kommt je Eintrag ein Verweis auf den Wert hinzu. Damit selten zwei Werte auf denselben Platz fallen, hält CPython einen Teil der Plätze frei und legt eine größere Tabelle an, sobald die alte zu voll wird. Die freien Plätze und die gespeicherten Hashwerte kosten Speicher. Das folgende Beispiel misst das für 100.000 Zahlen, jeweils nur den Container ohne die Zahlenobjekte. Die Werte können sich zwischen Python-Versionen leicht unterscheiden.

```
xs = list(range(100_000))      # 100.000 Zahlen als Liste
s = set(xs)                   # dieselben Zahlen als Set
d = dict.fromkeys(xs)         # ein Dict, das jede Zahl aus xs als Schlüssel enthält
for name, container in [("Liste", xs), ("Set", s), ("Dict", d)]:
    print(f"{name}: {sys.getsizeof(container) / 1e6:.1f} MB")
# Ausgabe: Liste: 0.8 MB, Set: 4.2 MB, Dict: 5.2 MB
```

Welcher Container für welche Aufgabe?

Die Wahl richtet sich nach der Operation, die am häufigsten vorkommt.

Häufigste Operation	Passender Container	Schreibweise	Zeit für diese Operation
Zugriff über die Position	Liste	<code>xs[i]</code>	unabhängig von der Länge der Liste
Zugriff auf einen Wert eines festen Pakets	Tupel	<code>hit[0]</code>	unabhängig von der Länge des Tupels
Nachschlagen über einen Schlüssel	Dict	<code>d[key]</code>	im Mittel unabhängig von der Größe des Dicts
Prüfen, ob ein Wert vorkommt	Set	<code>x in s</code>	im Mittel unabhängig von der Größe des Sets

Zum Vergleich: Die Prüfung `x in xs` in einer Liste braucht eine Zeit, die mit der Länge der Liste wächst. Auch das Auspacken eines Tupels, etwa `title, score = hit`, weist jeden Wert einzeln zu. Die Arbeit wächst dort mit der Anzahl der Werte.

Brauchen wir diese Container noch, wenn später NumPy und PyTorch rechnen? Ja, für andere Aufgaben. Die Zahlenblöcke, mit denen gerechnet wird, liegen später in Arrays und Tensoren. Was das Programm darum herum organisiert, bleibt in den Containern von Python. Beispiele sind die Einstellungen eines Trainingslaufs als Dict, die Liste der Dateien eines Datensatzes und das Vokabular eines Tokenizers als Dict, das jedem Token eine Nummer zuordnet. Auch PyTorch selbst liefert die Gewichte eines Modells als Dict, das jedem Namen einen Tensor zuordnet.

Für die nächste Aufgabe stellt die Vorgabe-Zelle alle Wörter des Korpus als eine lange Liste bereit. Die Texte werden dafür in Kleinbuchstaben umgewandelt und an Leerzeichen und Zeilenumbrüchen getrennt. Satzzeichen bleiben am Wort hängen, „stadt“ und „stadt.“ gelten also als verschiedene Wörter.

```
# Vorgabe: alle Wörter des Korpus als Liste – einfach ausführen
words = []
for p in passages:
    ① for w in p["text"].lower().split():
        words.append(w)
print(len(words), "Wörter")
```

① `lower()` wandelt einen Text in Kleinbuchstaben um, `split()` trennt ihn an Leerraum in eine Liste von Wörtern

59543 Wörter

Aufgabe 2.5.3 #1

5 min

Bilden Sie aus `words` ein Set `word_set`. Messen Sie dann mit `measure` und `search_many`, wie lange die Suche nach dem Wort "quantencomputer" dauert, das im Korpus nicht vorkommt: in `t_in_list` die Zeit je Suche in der Liste, in `t_in_set` die Zeit je Suche im Set. Orientieren Sie sich an der Messzelle oben.

Wer früher fertig ist: Wie viele verschiedene Wörter enthält der Korpus, und wie viele wären es ohne die Umwandlung in Kleinbuchstaben? Schätzen Sie erst, ob die Zahl steigt oder fällt. Bauen Sie dann die Wortliste in einer neuen Zelle ohne `lower()` auf und zählen Sie nach.

Hinweis: `measure(search_many, words, "quantencomputer", 10) / 10` misst zehn Suchen in der Liste und teilt durch zehn. Für das Set nehmen Sie 100.000 Wiederholungen.

```
# TODO: Ihre Lösung hier
...
```

```
# Prüfwelle – einfach ausführen
assert type(word_set) == set, "word_set soll ein Set sein"
assert len(word_set) == 19798, "das Set enthält jedes Wort genau einmal"
assert "quantencomputer" not in word_set
assert t_in_set < t_in_list, "die Suche im Set sollte deutlich schneller sein"
print(f"Liste: {t_in_list * 1e6:.1f} µs je Suche   Set: {t_in_set * 1e6:.3f} µs je Suche   Faktor
{t_in_list / t_in_set:,.0f}")
print("Alle Prüfungen bestanden ✓")
```

Lernfragen 2.5.3 #1

1. Lohnt es sich, das Set aufzubauen, wenn nur ein einziges Wort gesucht werden soll? Überlegen Sie, was der Aufbau kostet, und messen Sie ihn in der Zelle darunter.

```
t_build_set = measure(set, words)
t_one_search = measure(search_many, words, "quantencomputer", 1)
print(f"Set aufbauen: {t_build_set * 1000:.2f} ms   eine Suche in der Liste: {t_one_search * 1000:.2f} ms")
```

Set aufbauen: 1.68 ms eine Suche in der Liste: 0.25 ms

2.6 Zahlen im Rechner

M2 · Data-Science-Werkzeugkasten

Kurz überlegen

Was liefert $0.1 + 0.2 == 0.3$? Legen Sie sich fest, bevor Sie die nächste Zelle ausführen.

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)

total = 0.0
for _ in range(10):
    total += 0.1          # zehnmal 0.1 addieren
print(total)
```

```
0.30000000000000004
False
0.9999999999999999
```

Der Rechner speichert Fließkommazahlen binär, als Summe von Zweierpotenzen. Die meisten Dezimalbrüche lassen sich so nicht exakt darstellen. Gespeichert wird die nächstgelegene darstellbare Zahl, und jede Rechnung rundet erneut.

Im Dezimalsystem kennen Sie dasselbe von $1/3$: Die Ziffernfolge 0,333... bricht nie ab, jede Darstellung mit endlich vielen Stellen ist gerundet. Im Binärsystem trifft es schon 0,1. Die gespeicherte Zahl liegt minimal daneben, und bei jeder Rechnung können sich solche Abweichungen fortpflanzen. Das ist kein Fehler von Python, in C++ verhalten sich `double` und `float` genauso.

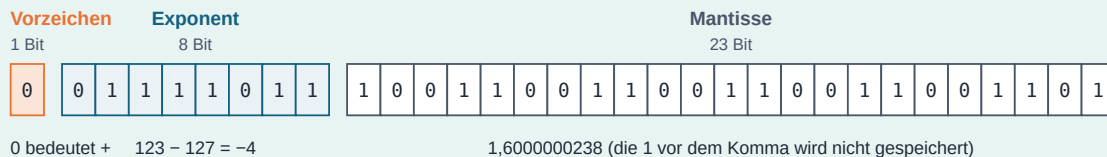
Vertiefung: Wie ist eine Fließkommazahl gespeichert?

NICHT PRÜFUNGSRELEVANT

Die üblichen Formate folgen dem Standard IEEE 754 und teilen ihre Bits in drei Felder für Vorzeichen, Exponent und Mantisse. Der Wert ergibt sich als Vorzeichen mal Mantisse mal 2 hoch Exponent. Das ist die wissenschaftliche Schreibweise, die Sie von $6,02 \cdot 10^{23}$ kennen, nur zur Basis 2.

Ein Bit hält das Vorzeichen. Der Exponent legt die Größenordnung als Zweierpotenz fest. Er wird mit einem festen Versatz gespeichert, damit auch negative Exponenten darstellbar sind: Bei `float32` wird vom gespeicherten Wert 127 abgezogen. Die Mantisse hält die Ziffern der Zahl, und ihre Länge begrenzt die Genauigkeit. Die Abbildung zeigt die 32 Bit, mit denen `float32` die Zahl 0.1 speichert.

0.1 als float32: 32 Bit in drei Feldern



$$\text{Wert} = \text{Vorzeichen} \cdot \text{Mantisse} \cdot 2^{\text{Exponent}} = + 1,6000000238 \cdot 2^{-4} = 0,10000000149...$$

float64 ist genauso aufgebaut: 1 Bit Vorzeichen, 11 Bit Exponent, 52 Bit Mantisse.

Zwei Formate für Fließkommazahlen

Zwei Formate sind üblich. Sie unterscheiden sich in Speicherbedarf und Genauigkeit.

Format	Speicher für den Zahlenwert	ungefähre Genauigkeit	wo
<code>float64</code>	8 Bytes	15 bis 16 signifikante Dezimalziffern	der Python- <code>float</code> , in C++ <code>double</code>
<code>float32</code>	4 Bytes	6 bis 7 signifikante Dezimalziffern	in C++ <code>float</code> , Standard in KI-Bibliotheken

Die Zahl der Bits begrenzt die Genauigkeit. Signifikante Ziffern sind die Ziffern einer Zahl ab der ersten Ziffer, die nicht 0 ist. Die Position des Kommas spielt dafür keine Rolle: 123456,7 und 0,001234567 haben jeweils sieben signifikante Ziffern. Durch Rechnungen kann weitere Genauigkeit verloren gehen. Die 8 Bytes sind der reine Zahlenwert. Als Python-Objekt belegt eine solche Zahl 24 Bytes, wie im Kapitel [2.5.2](#) gemessen.

Exkurs: Datentypen, die Dezimalbrüche exakt speichern

NICHT PRÜFUNGSRELEVANT

Es gibt Datentypen, die Dezimalbrüche exakt speichern. Sie legen die Dezimalziffern selbst ab statt einer binären Näherung. Python bringt dafür `decimal.Decimal` mit, außerdem `fractions.Fraction` für exakte Brüche. Java kennt `BigDecimal`, C# den Typ `decimal`, Datenbanken kennen `DECIMAL` und `NUMERIC`. Eingesetzt werden diese Typen vor allem für Geldbeträge.

Der Preis: Diese Typen rechnen in Software statt mit den Fließkomma-Befehlen des Prozessors und sind deshalb langsamer. Sie belegen mehr Speicher, und Grafikprozessoren bieten für sie keine Rechenbefehle. Auch diese Typen haben Grenzen. `Decimal` speichert endliche Dezimalzahlen exakt. Der Bruch $1/3$ hat keine endliche Dezimaldarstellung und wird bei `Decimal` auf die eingestellte Stellenzahl gerundet. `Fraction` speichert Zähler und Nenner und stellt deshalb auch $1/3$ exakt dar. Irrationale Zahlen wie die Wurzel aus 2 kann keiner der beiden Typen exakt darstellen. Für das numerische Rechnen mit Millionen von Werten sind Fließkommazahlen deshalb üblich, und die kleinen Abweichungen werden mit Toleranzen abgefangen.

```

from decimal import Decimal
from fractions import Fraction
① print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
② print(Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10))
print(Decimal(1) / Decimal(3))           # Decimal rundet 1/3 auf die eingestellte Stellenzahl
print(Fraction(1, 3) * 3 == 1)           # Fraction stellt 1/3 exakt dar

```

- ① `Decimal("0.1")` speichert die Dezimalziffern der Zahl exakt, angegeben wird sie als Text
 ② `Fraction(1, 10)` speichert den Bruch 1/10 mit Zähler und Nenner

```

True
True
0.333333333333333333333333333333333333
True

```

Die Kursvektoren sind als `float32` gespeichert. Das halbiert den Speicherbedarf, und GPUs rechnen in diesem Format und in noch kleineren Formaten am schnellsten. Was die geringere Genauigkeit bedeutet, zeigt ein `array.array` mit 32-Bit-Zahlen. Es speichert 0.1 und liefert beim Auslesen einen sichtbar anderen Wert zurück. Das zweite Beispiel ist eine ganze Zahl. Bis einschließlich $2^{24} = 16.777.216$ stellt `float32` jede ganze Zahl exakt dar. Direkt darüber beträgt der Abstand zwischen zwei darstellbaren Zahlen 2. Die 16.777.217 liegt dazwischen und wird hier auf 16.777.216 gerundet.

```

a = array.array("f", [0.1, 16777217.0]) # zwei Zahlen als 32-Bit-Zahlen speichern
print(a[0])                            # 0.1 kommt sichtbar verändert zurück
print(a[1])                            # oberhalb von 2**24 liegen darstellbare Zahlen im Abstand 2

```

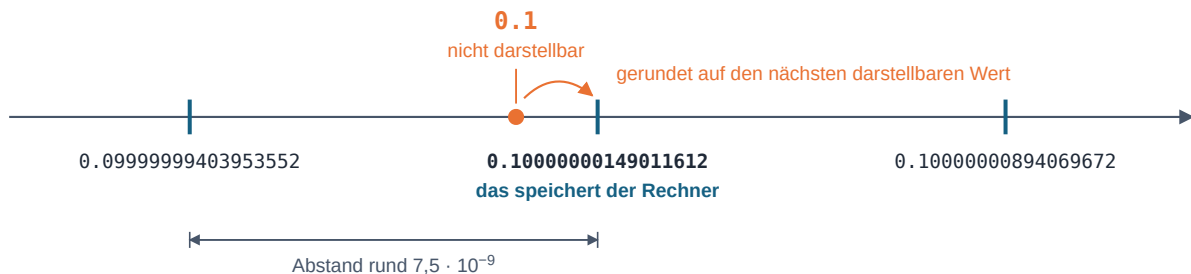
```

0.10000000149011612
16777216.0

```

float32 in der Umgebung von 0.1

darstellbar sind nur die markierten Werte, dazwischen gibt es nichts



`float64` folgt demselben Prinzip. Das Raster ist dort bei 0.1 rund 537 Millionen Mal feiner (Abstand rund $1,4 \cdot 10^{-17}$), aber auch dort liegt 0.1 zwischen zwei darstellbaren Werten.

Exkurs: Quantisierung von Sprachmodellen

(NICHT PRÜFUNGSRELEVANT)

Die Gewichte eines Sprachmodells sind Fließkommazahlen, und ihr Format bestimmt den Speicherbedarf. Ein Modell mit 8 Milliarden Gewichten belegt in `float32` rund 32 GB, in einem Format mit 16 Bit rund 16 GB. Quantisierung geht weiter und speichert jedes Gewicht als kleine ganze Zahl mit 8 oder 4 Bit, zusammen mit einem Umrechnungsfaktor je Gruppe von Gewichten. Dasselbe Modell kommt dann mit rund 8 GB oder 4 GB aus.

Der Preis ist Genauigkeit: Mit 4 Bit lassen sich nur 16 verschiedene Werte unterscheiden. Die Antworten des Modells können dadurch schlechter werden. Wie stark, hängt von Modell, Verfahren und Aufgabe ab. Quantisiert wird in der Regel ein fertig trainiertes Modell für den Betrieb. Trainiert wird meist in `float32` oder in einer Mischung aus 16 und 32 Bit. Der Speicherbedarf entscheidet darüber, ob ein Modell in den Speicher einer Grafikkarte passt. Darauf kommen wir später im Semester zurück.

Jetzt lässt sich erklären, warum die Prüfwellen dieses Notebooks einen Abstand prüfen. Schon ein normalisierter Vektor hat selten exakt die Länge 1:

```
print(norm(normalize([3.0, 4.0])))
print(norm(normalize([1.0, 1.0])))

exact = 0
for v in vectors:
    if norm(normalize(v)) == 1.0:
        exact += 1
print(exact, "von", len(vectors), "normalisierten Absatzvektoren haben exakt die Länge 1.0")
```

```
1.0
0.9999999999999999
49 von 400 normalisierten Absatzvektoren haben exakt die Länge 1.0
```

Vergleichen mit Toleranz. Wer Ergebnisse von Rechnungen mit Fließkommazahlen vergleicht, legt fest, welche Abweichung noch als gleich gilt. Das einfachste Muster ist der absolute Abstand: `abs(a - b) <= tol`, wobei `tol` für die Toleranz steht. Die Standardbibliothek bietet dafür `math.isclose(a, b)`. Ohne weitere Angaben prüft diese Funktion einen relativen Abstand: Die Zahlen müssen in etwa neun Stellen übereinstimmen. Beim Vergleich mit 0 hilft das nicht, dort gibt man mit `abs_tol` zusätzlich einen absoluten Abstand an. Welche Toleranz angemessen ist, hängt von der Rechnung ab. Für die kleinen Rechnungen mit 64-Bit-Zahlen in diesem Notebook genügt `1e-9`. Beim Vergleich mit den 32-Bit-Werten aus NumPy wäre das zu streng, dort stimmen nur sechs bis sieben Ziffern.

Aufgabe 2.6 #1

5 min

Schreiben Sie die Funktion `is_unit(v, tol=1e-9)`. Sie prüft, ob `v` ein Einheitsvektor ist, also die Länge 1 hat. Der Parameter `tol` steht für die Toleranz: Die Funktion gibt `True` zurück, wenn die Länge von `v` höchstens um `tol` von 1 abweicht, sonst `False`. Notieren Sie darunter als Kommentar in einem Satz, warum `norm(v) == 1.0` für diesen Test ungeeignet ist.

Wer früher fertig ist: Finden Sie einen Vektor, für den `is_unit` mit der Standardtoleranz `False` liefert und mit `tol=0.01` den Wert `True`. Seine Länge muss dafür knapp neben 1 liegen. Prüfen Sie beide Aufrufe in einer neuen Zelle.

Hinweis: Die Abweichung kann nach oben oder nach unten gehen. Der Betrag `abs(...)` erfasst beide Richtungen.

```
def is_unit(v, tol=1e-9):
    # TODO: Ihre Lösung hier
    ...

# Warum ist norm(v) == 1.0 als Test ungeeignet?
# TODO: Ihre Lösung hier
...
```

```
# Prüfwerte – einfach ausführen
assert is_unit([1.0, 0.0]) == True, "ein Vektor der Länge 1"
assert is_unit(normalize([1.0, 1.0])) == True, "0.9999999999999999 gilt als Länge 1"
assert is_unit(normalize(vectors[0])) == True, "ein normalisierter Absatzvektor"
assert is_unit([3.0, 4.0]) == False, "Länge 5"
assert is_unit([1.001, 0.0]) == False, "ein Tausendstel zu lang ist bei tol=1e-9 zu viel"
assert is_unit([0.999, 0.0]) == False, "ein Tausendstel zu kurz ebenfalls (Betrag)"
assert is_unit([1.001, 0.0], tol=0.01) == True, "tol muss berücksichtigt werden"
assert is_unit([1.125, 0.0], tol=0.125) == True, "eine Abweichung von genau tol gilt noch"
print("Alle Prüfungen bestanden ✓")
```

Der Nullvektor und `eps`. Eine Vereinbarung vom Anfang ist noch offen: Was geschieht mit einem Vektor der Länge 0? `normalize([0.0, 0.0])` teilt durch null und bricht in reinem Python mit `ZeroDivisionError: float division by zero` ab. Ein verbreiteter Ausweg ist ein kleiner positiver Wert unter der Wurzel. Er heißt üblicherweise `eps`, kurz für Epsilon. Mit diesem Buchstaben bezeichnet die Mathematik eine sehr kleine positive Zahl. Der Nenner kann dann nie null werden. Die folgende Variante heißt zur Unterscheidung `normalize_eps` und verwendet `eps = 1e-6`.

```
def normalize_eps(v, eps=1e-6):
    total = 0.0
    for x in v:
        total += x * x
    length = math.sqrt(total + eps)      # eps unter der Wurzel: nie null
    result = []
    for x in v:
        result.append(x / length)
    return result

print(normalize_eps([0.0, 0.0]))        # der Nullvektor bleibt ein Nullvektor
print(normalize_eps([3.0, 4.0]))
print(is_unit(normalize_eps([3.0, 4.0])))
print(norm(normalize_eps([0.0001, 0.0]))) # ein sehr kurzer Vektor
```

```
[0.0, 0.0]
[0.5999999880000003, 0.7999999840000004]
False
0.09950371902099892
```

Der Nullvektor bleibt ein Nullvektor, und kein Aufruf bricht mehr ab. Das hat einen Preis: `[3, 4]` wird nicht mehr zu `[0.6, 0.8]`, die Länge des Ergebnisses weicht um rund zwei Hundertmillionstel von 1 ab. `is_unit` meldet deshalb mit der Standardtoleranz `False`. Diese Abweichung ist kein Rundungsfehler. Sie folgt aus der geänderten Formel und tritt bei jeder Eingabe auf. Wie groß sie ist, hängt von der Länge des Vektors ab. Bei unseren Absatzvektoren mit Längen zwischen 1,8 und 4,1 ist sie winzig. Der sehr kurze Vektor in der letzten Zeile hat nach `normalize_eps` dagegen nur die Länge 0,0995 und ist damit weit von einem Einheitsvektor entfernt. Das Skalarprodukt zweier so behandelter Vektoren ist dann nicht mehr die Kosinus-Ähnlichkeit. Wer `eps` einführt, muss wissen, in welchem Größenbereich seine Vektoren liegen, und seine Toleranzen daran ausrichten. Später im Semester wird genau diese Fassung mit `eps` zur verbindlichen Vereinbarung für die Normalisierung.

Vertiefung: Wird das in der Praxis so gemacht?

NICHT PRÜFUNGSRELEVANT

Ja, ein `eps` ist in numerischen Bibliotheken üblich. PyTorch teilt beim Normalisieren durch das Maximum aus Länge und `eps`. Normalisierungsschichten in neuronalen Netzen addieren `eps` unter der Wurzel, so wie die Funktion oben.

Eine Prüfung auf null mit `if` wäre in unserer Python-Funktion genauso geeignet. Auch auf ganzen Zahlenblöcken sind Fallunterscheidungen möglich, in PyTorch zum Beispiel mit `torch.where`. Ein `eps` leistet zusätzlich zwei Dinge. Es fängt sehr kleine Nenner ab und nicht nur die exakte Null. Und es legt ein bestimmtes Verhalten fest: Der Nullvektor bleibt ein Nullvektor, sehr kurze Vektoren bleiben kurz. Das kann erwünscht sein. Wenn ein Vektor überwiegend Rundungsfehler enthält, vergrößert das Normalisieren ohne `eps` diese Fehler auf die Länge 1. Ein kleiner Betrag allein sagt darüber allerdings nichts aus.

Lernfragen 2.6 #2

- Ein Test vergleicht ein berechnetes Ergebnis mit `==` gegen den Sollwert `0.3` und schlägt fehl, obwohl die Rechnung fachlich stimmt. Was ist die wahrscheinliche Ursache, und wie sollte der Test aussehen?
- Welche Aussagen über `float32` und `float64` stimmen?
 - Ein `float32`-Wert belegt halb so viel Speicher wie ein `float64`-Wert.
 - `float32` stellt Dezimalbrüche wie 0.1 exakt dar, `float64` nicht.
 - Der Python-`float` entspricht `float64`.
 - Mit `float64` sind Vergleiche mit `==` nach Rechnungen unbedenklich.
- `normalize_eps` liefert für `[3.0, 4.0]` einen Vektor, dessen Länge um rund zwei Hundertmillionstel von 1 abweicht. Ist das ein Rundungsfehler?

→ Nächste Schritte

- Beenden Sie die Aufgaben, die im Termin offen geblieben sind. Zur Schlusskontrolle starten Sie die Laufzeit neu und führen alle Zellen aus: Alle Prü fzellen sollten mit Ihren eigenen Lösungen durchlaufen.
- Bearbeiten Sie Teil 2 des Python-Grundkurses (Container und Daten). Er behandelt Tupel, Dictionaries, Sets, Slicing und Comprehensions in voller Breite.
- Die Lösungen zur letzten Lerneinheit stehen in Moodle bereit.

→ **Ausblick:** In der nächsten Lerneinheit übernimmt NumPy. Sie schreiben dieselbe Suche als Rechnung auf ganzen Zahlenblö cken und messen sie gegen Ihre Fassung von heute.

Quellen und Referenzen

- Python Software Foundation: [Python-Tutorial](#), Kapitel 5 (Datenstrukturen), Kapitel 9 (Klassen) und Kapitel 15 (Gleitkomma-Arithmetik: Probleme und Einschränkungen).
- Python Software Foundation: [Datenmodell](#) (Sondermethoden wie `__len__`, `__getitem__`, `__repr__`) sowie die Bibliotheksdokumentation zu [sys.getsizeof](#), [array](#) und [math.isclose](#).
- Datengrundlage: [GermanQuAD](#) (deepset), Lizenz CC BY 4.0. Die Absatztexte stammen aus der deutschen Wikipedia (CC BY-SA). Attribution auch in `data/appetizer_passages.json`.

Dieses Material wurde mit Unterstützung von KI-Werkzeugen entwickelt (Text und Code: Claude und ChatGPT, Karikaturen: Google Gemini). Alle Inhalte sind von Prof. Dr. Stephan Kurpjuweit konzipiert, geprüft und verantwortet.